

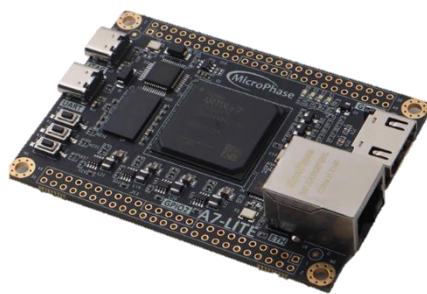
基於 AMD Artix-7 FPGA 之 XADC 晶片溫度監測 與 UART 即時傳輸設計

一、摘要

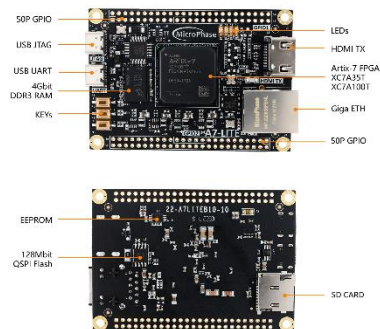
本實驗以 AMD Artix-7 FPGA 開發板作為實作平台，板載核心為 XC7A200T，Artix-7 系列資源較豐富的 FPGA 晶片。它採用 28nm 工藝製造。

實驗設計一套晶片溫度與板載狀態資料之即時監測系統。FPGA 端透過內部 XADC 取得晶片接面溫度原始資料，並整合按鍵狀態、LED 狀態與封包計數器，將資料封裝為 ASCII 文字封包後，透過 UART TX 傳送至電腦端。

電腦端使用 Python 與 pyserial 建立即時接收程式，負責解析 UART 封包、擷取 CNT、KEY、LED 與 TEMP 欄位，並將 XADC raw data 轉換為攝氏溫度。實驗結果顯示，系統能週期性傳送 FPGA 內部溫度與板載狀態資料，並可透過 Vivado System Monitor 進行溫度數值比對驗證。本專題完成 FPGA 端感測資料擷取、UART 封包傳輸、Python 即時監控與上板驗證流程，可作為後續 FPGA 系統除錯、硬體狀態監控與嵌入式通訊設計之基礎架構。

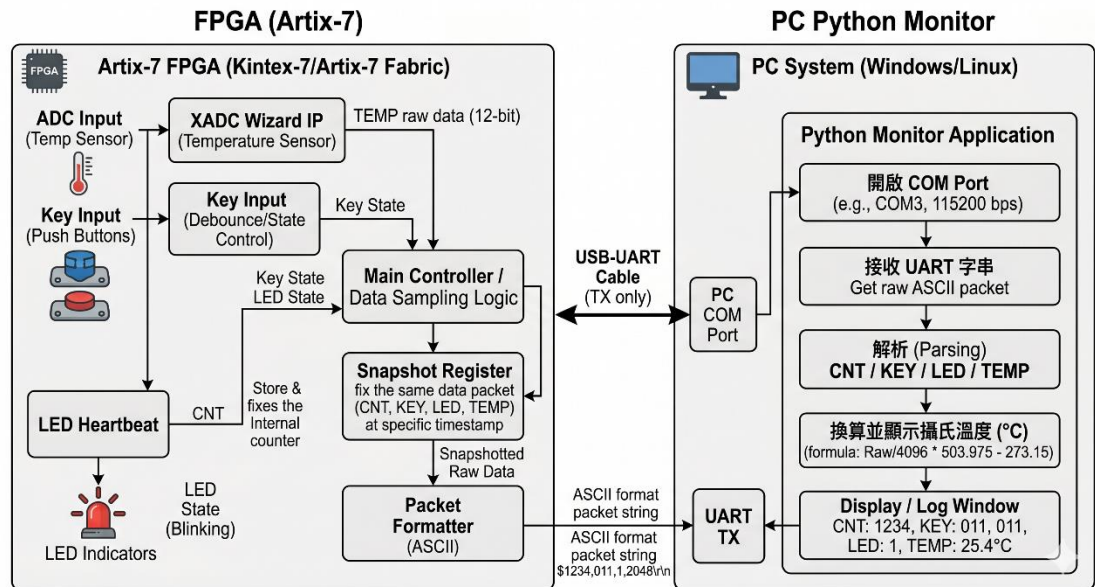


功能接口



二、設計方向

本實驗分為 FPGA 端與電腦端兩部分。FPGA 端負責資料產生、溫度讀取、封包組成與 UART 傳送；電腦端負責 UART 接收、文字解析與溫度換算。



(一) FPGA 模組

- [top_uart_temp_monitor.v](#) : 系統頂層，整合 UART、XADC、LED、按鍵與封包產生邏輯。
- [uart_tx.v](#) : UART 傳送模組，負責將 8-bit 資料轉換成序列位元輸出。
- [xadc_temp_reader.v](#) : XADC 溫度讀取模組，透過 DRP 介面取得溫度 raw data。
- [xadc_wiz_1](#) : Vivado XADC Wizard IP，用於存取 FPGA 內部 XADC。
- [tb_top_uart_temp_monitor.v](#) : Testbench，用於功能模擬與波形驗證。

(二) 電腦程式架構

- 開啟指定 COM Port。
- 設定 UART baud rate 為 115200。
- 逐行讀取 FPGA 傳來的文字封包。
- 使用正規表示式解析 CNT、KEY、LED、TEMP 欄位。
- 將 XADC raw data 轉換為攝氏溫度。

- 即時顯示監控資訊。

(三) 名詞解釋

- UART (Universal Asynchronous Receiver-Transmitter) 是一種常見的非同步串列通訊協定。tx 代表 Transmit (傳送)。
- XADC 是 AMD/Xilinx FPGA 晶片內建的類比數位轉換器 (Analog-to-Digital Converter)。DRP (Dynamic Reconfiguration Port) 是 FPGA 內部用來讀寫硬體暫存器的一種通訊介面。
- IP (Intellectual Property Core) 在 FPGA 領域指的是「現成的硬體模組」。
- Baud Rate 是設備之間「溝通的速度」。UART 的「A」代表 Asynchronous (非同步)。這意味著傳送端 (FPGA) 和接收端 (電腦) 之間，沒有一根共同的「時脈線 (Clock)」來對時。既然沒有時脈，電腦就不知道 FPGA 什麼時候丟了一個 0，什麼時候丟了一個 1。所以在雙方在通訊前，先「約定好相同的溝通速度 (Baud Rate)」。

三、FPGA RTL 程式

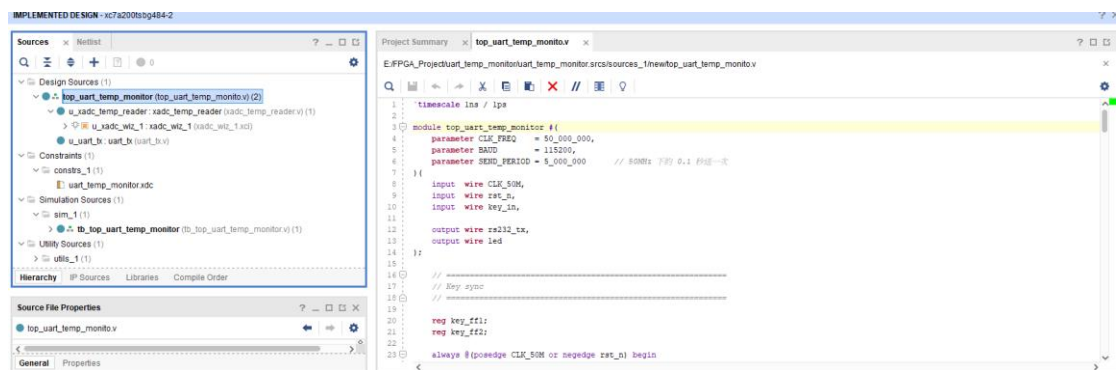
(一) 模組架構

`top_uart_temp_monitor.v` 設定為系統頂層。

--- 程式詳情請參考 [GitHub](#) ---

UART 是非同步序列傳輸介面，不需額外傳送時鐘訊號。傳送端與接收端必須事先設定相同的 baud rate。本專題採用 115200 baud，資料格式為常見的 8N1，即 1 個起始位元、8 個資料位元、無同位元、1 個停止位元。

UART TX 在空閒狀態時維持高電平。當開始傳送一個 byte 時，TX 會先送出低電平起始位元，接著由低位元至高位元依序送出 8-bit 資料，最後送出高電平停止位元。本實驗的 `uart_tx.v` 模組透過 baud counter 產生每個 bit 的時間間隔，並以 shift frame 的方式依序輸出 start bit、data bits 與 stop bit。



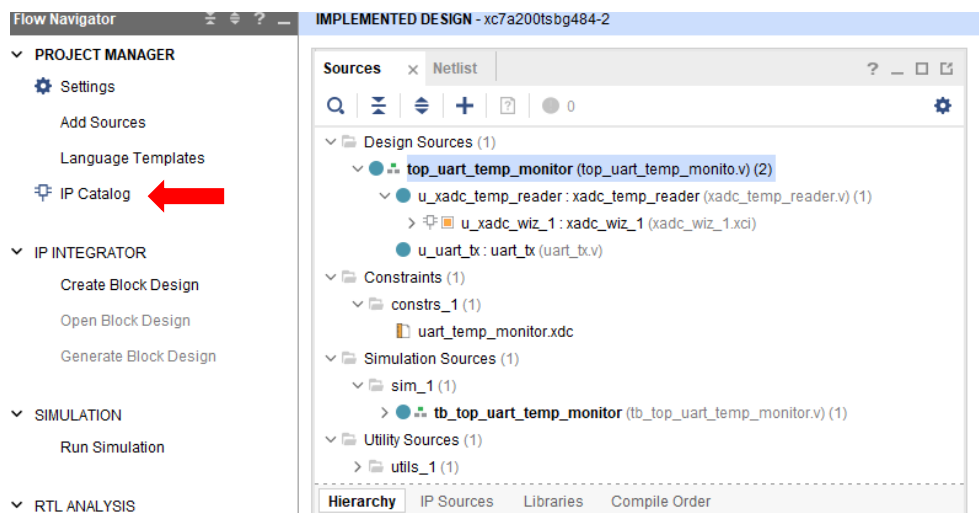
(二) XADC 溫度讀取原理

Artix-7 FPGA 內部具有 XADC，可用於讀取內部感測資料。

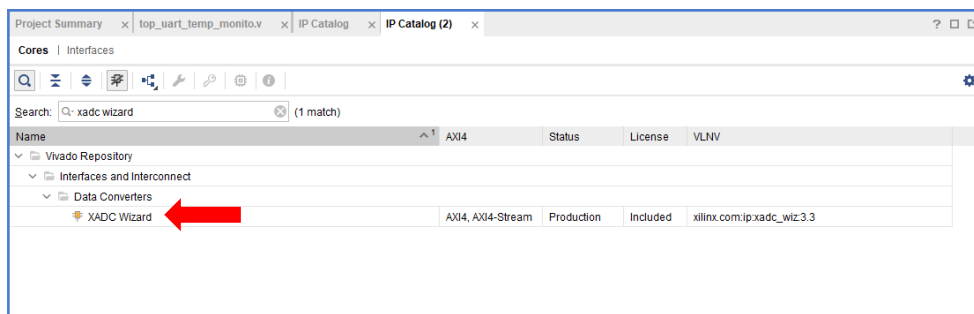
實驗使用 Vivado 的 XADC Wizard IP，設定為 DRP 介面與 Continuous Mode，並選擇 Temperature channel 作為讀取目標。

方法：

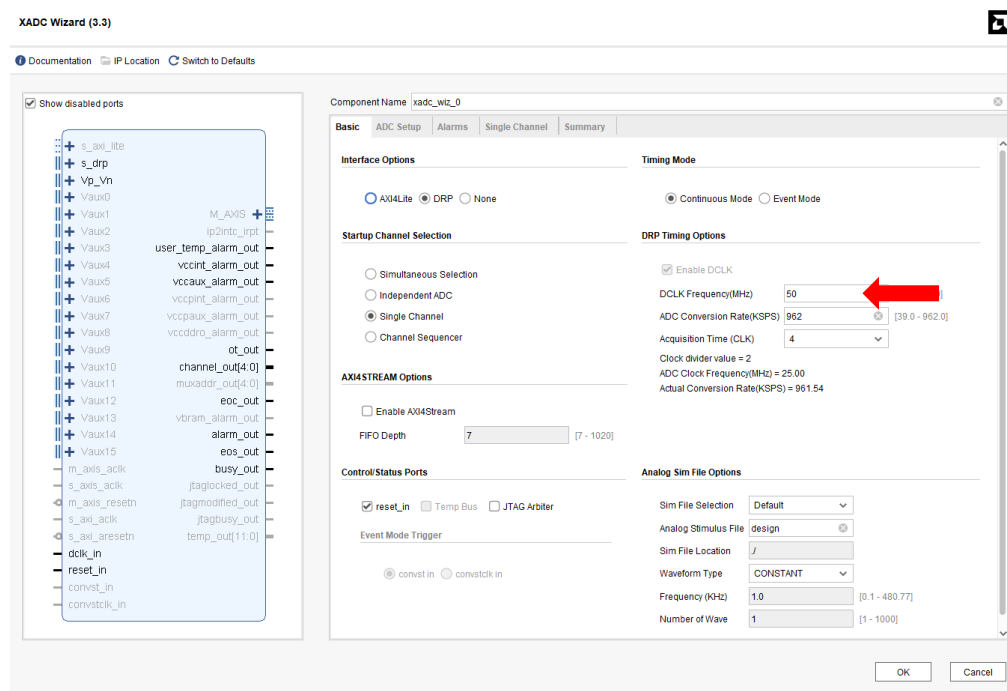
Step1. 在 Vivado 左側工作列選擇 IP Catalog。



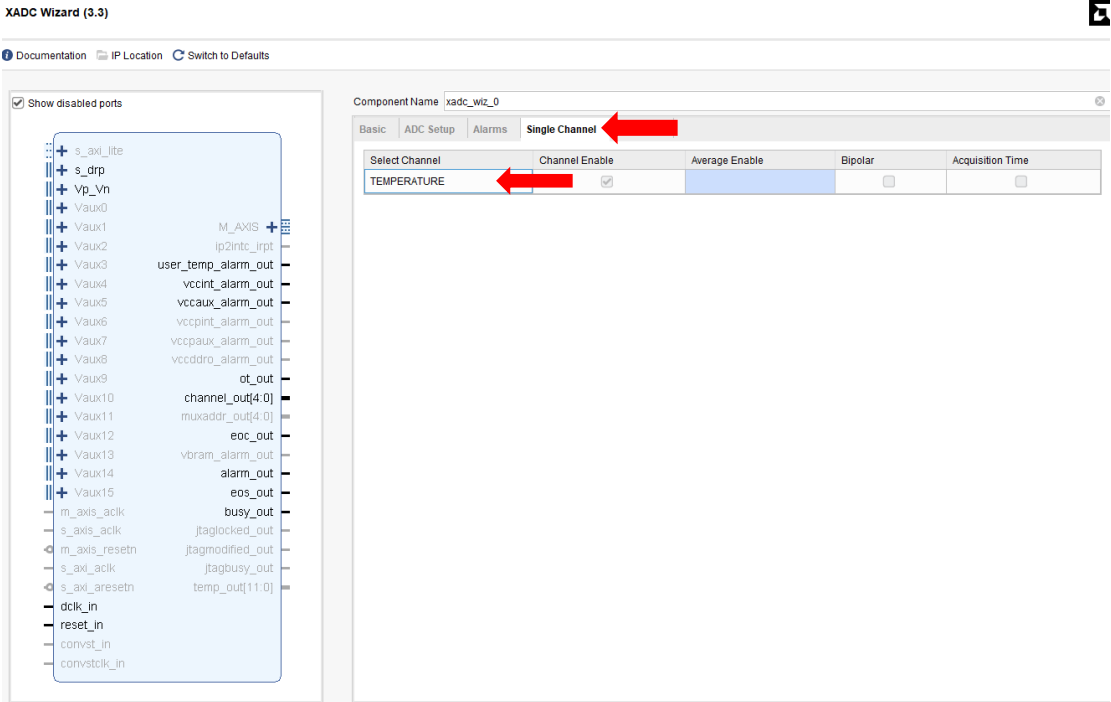
Step2. 在該頁面搜尋欄中搜尋「XADC Wizard」



Step3. DRP Timing Options 中選擇「DCLK Frequency」將頻率調整成 50MHz



Step4.Single Channel 中選擇「TEMPERATURE」



XADC 輸出的溫度資料為 16-bit raw data，其中有效 ADC code 位於高 12-bit。Python 程式取得 TEMP=0xFFFF 後，先將 raw data 右移 4 bit 取得 12-bit ADC code，再套用溫度轉換公式：

$$\text{Temperature} = \text{ADC_Code} \times 503.975 / 4096 - 273.15$$

例如當 FPGA 傳回：TEMP=0x97A0 代表 raw data 為 0x97A0，有效 ADC code 為 0x97A，可換算為約 25°C 左右的晶片接面溫度。

(三) 監控資料流

`top_uart_temp_monitor.v` 負責整合所有功能。系統會以固定週期產生一包監控資料，內容包含：

欄位	說明
CNT	封包計數器
KEY	按鍵狀態
LED	LED 心跳狀態
TEMP	XADC 溫度 raw data

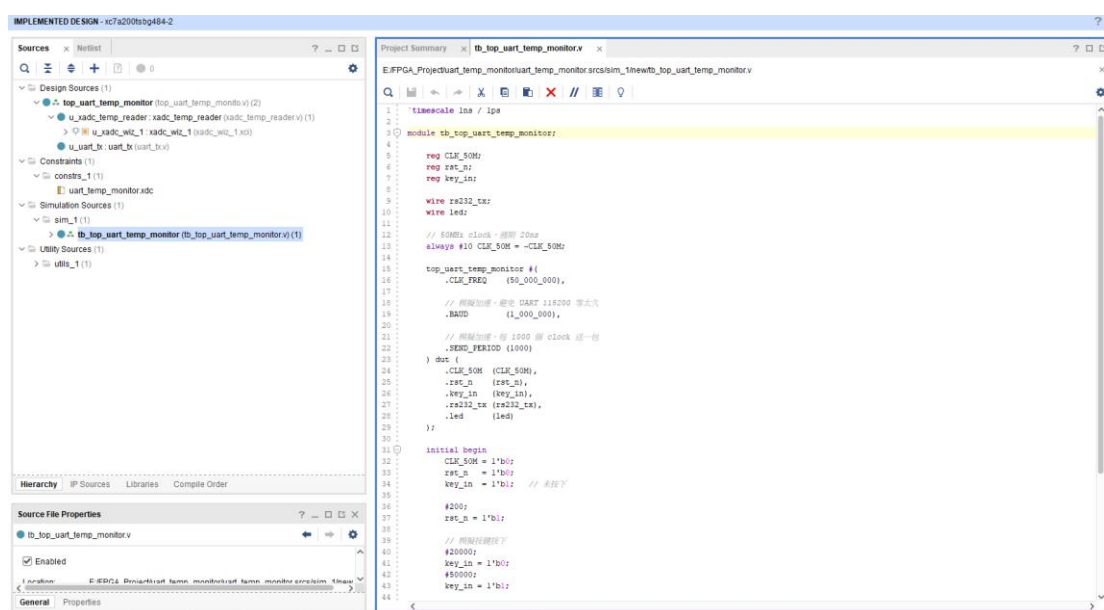
系統在準備傳送封包前，會先將當下的狀態存入 snapshot register，避免封包傳送過程中資料變動造成內容不一致。

四、撰寫測試平台

(一)撰寫 Testbench (`tb_top_uart_temp_monitor.v`)

--- 程式詳情請參考 *GitHub* ---

使用 `tb_top_uart_temp_monitor.v` 進行功能模擬。Testbench 產生 50MHz clock，並模擬 reset 與按鍵輸入變化。為了縮短模擬時間，模擬時可將 UART baud rate 提高至 1Mbps，並縮短封包傳送週期。模擬時主要觀察以下訊號：



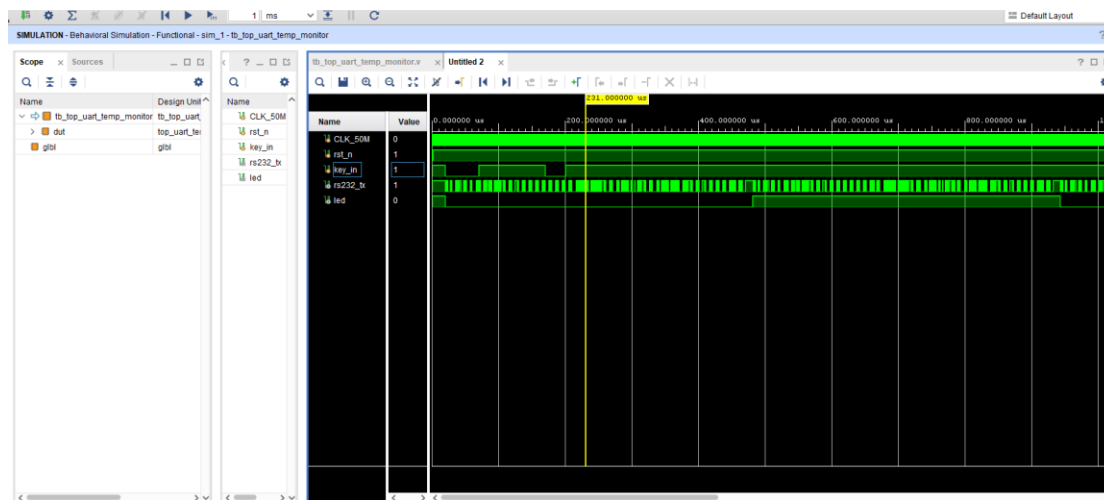
訊號	預期現象
CLK_50M	週期性跳變，週期 20ns
rst_n	初始為 0，之後解除為 1
rs232_tx	空閒為 1，傳送時出現起始位與資料跳變
tx_data	依序出現 ASCII code，例如 46 50 47 41 代表 FPGA
tx_start	每送一個 byte 產生一個 pulse
tx_busy	UART 傳送期間為 1
byte_idx	從 0 增加至封包最後一個 byte

若 tx_data 依序出現 46 50 47 41，即可確認封包開頭為 FPGA，代表 UART 封包產生邏輯正確。

(二)模擬訊號

由模擬波形可觀察到，rs232_tx 在空閒狀態時維持高電位。當

tx_start 產生 pulse 後，tx_busy 進入高電位，表示 UART TX 模組開始傳送資料。tx_data 依序輸出 ASCII code，例如 46 50 47 41，對應字元為 FPGA，可確認封包開頭產生正確。byte_idx 由 0 遞增至封包最後一個 byte，代表封包資料依序送入 UART TX 模組進行序列傳輸。



五、約束檔案與上板驗證

(一)約束檔案

```
##### 50MHz Clock #####
create_clock -period 20.000 -name CLK_50M [get_ports CLK_50M]

set_property PACKAGE_PIN J19 [get_ports CLK_50M]
set_property IOSTANDARD LVCMOS33 [get_ports CLK_50M]

##### Reset #####
set_property PACKAGE_PIN L18 [get_ports rst_n]
set_property IOSTANDARD LVCMOS33 [get_ports rst_n]

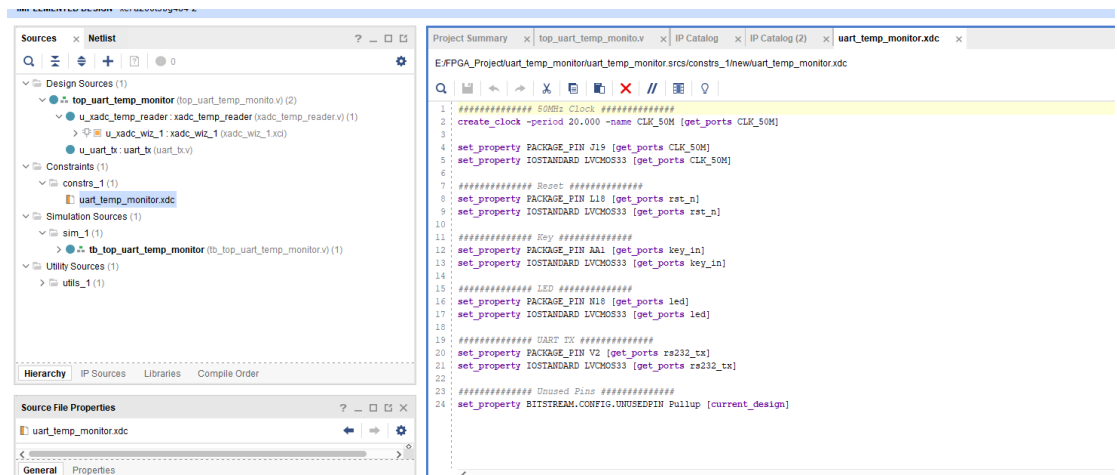
##### Key #####
set_property PACKAGE_PIN AA1 [get_ports key_in]
set_property IOSTANDARD LVCMOS33 [get_ports key_in]

##### LED #####
set_property PACKAGE_PIN N18 [get_ports led]
set_property IOSTANDARD LVCMOS33 [get_ports led]

##### UART TX #####
set_property PACKAGE_PIN V2 [get_ports rs232_tx]
set_property IOSTANDARD LVCMOS33 [get_ports rs232_tx]

##### Unused Pins #####

set_property BITSTREAM.CONFIG.UNUSEDPIN Pullup [current_design]
```

(二)驗證流程

本實驗使用 50MHz 系統時鐘，並將 UART TX、reset、按鍵與 LED 綁定至對應 FPGA 腳位。約束檔需包含 clock constraint 與 I/O standard 設定。

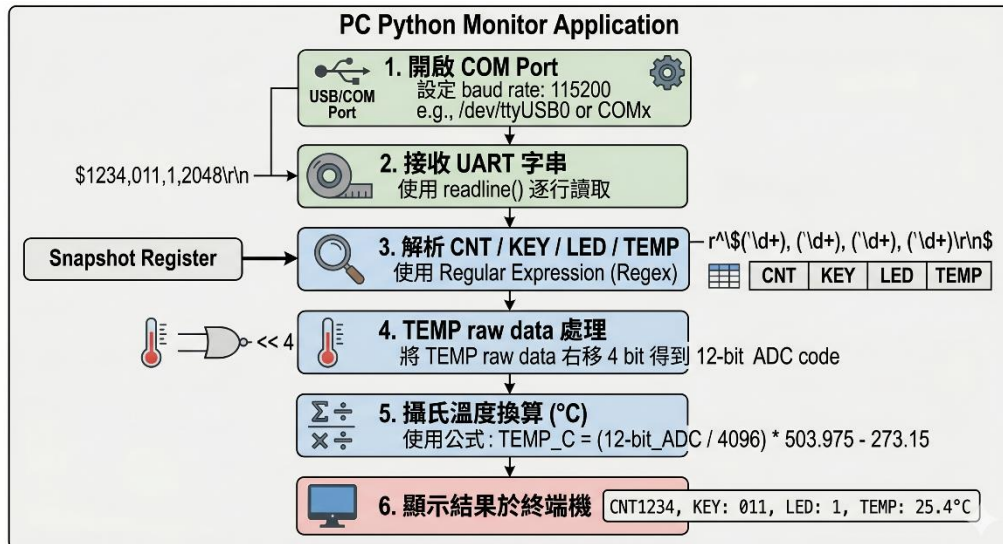
上板驗證流程如下：

1. Run Synthesis
2. Run Implementation
3. 檢查 Timing Summary
4. Generate Bitstream
5. Open Hardware Manager
6. Program Device
7. 執行 Python 監控程式
8. 觀察 UART 輸出與溫度換算結果
9. Python 端會持續顯示封包計數、按鍵狀態、LED 狀態與溫度。

六、Python 程式與電腦端接收

(一)Python 程式

--- 程式詳情請參考 *GitHub* ---



(二)電腦端接收

Vivado 模擬結果顯示 UART TX 會週期性產生序列資料，LED 訊號也會隨封包傳送週期翻轉。上板後，Python 端可透過 UART 接收 FPGA 傳來的監控封包，並將溫度 raw data 轉換為攝氏溫度。

開啟 COM Port。設定 baud rate 為 115200。使用 readline() 逐行讀取 UART 字串。使用 regular expression 解析 CNT、KEY、LED、TEMP。將 TEMP raw data 右移 4 bit 得到 12-bit ADC code。使用公式換算攝氏溫度。將結果顯示於終端機。

範例輸出如下：

```

uart_monitor.py x top_uart_temp_monito.v
uart_monitor.py > main
99 def main():
145     # 如果格式不符合，原樣印出，方便 debug
146     print(f"[{now}] RAW: {line}")
147     else:
148         # 格式化顯示結果
149         print(
150             f"[{now}] "
151             f"CNT={data['cnt']:08d} | "
152             f"KEY={data['key']:04d} | "
  
```

```

=====
UART FPGA Temperature Monitor
=====
Port : COM5
Baud : 115200
Press Ctrl+ C to stop.
=====
UART connected.

[23:23:35] CNT=00067572 | KEY=0 | LED=0 | TEMP_RAW=0x0C86 | TEMP=34.94 °C
[23:23:36] CNT=00067573 | KEY=0 | LED=1 | TEMP_RAW=0x0C8E | TEMP=34.57 °C
[23:23:36] CNT=00067574 | KEY=0 | LED=0 | TEMP_RAW=0x0C8A | TEMP=34.70 °C
[23:23:36] CNT=00067575 | KEY=0 | LED=1 | TEMP_RAW=0x0C8C | TEMP=35.31 °C
[23:23:36] CNT=00067576 | KEY=0 | LED=0 | TEMP_RAW=0x0C84 | TEMP=34.57 °C
[23:23:36] CNT=00067577 | KEY=0 | LED=1 | TEMP_RAW=0x0C62 | TEMP=34.70 °C
[23:23:36] CNT=00067578 | KEY=0 | LED=0 | TEMP_RAW=0x0C84 | TEMP=34.94 °C
[23:23:36] CNT=00067579 | KEY=0 | LED=1 | TEMP_RAW=0x0C7A | TEMP=34.82 °C
[23:23:36] CNT=00067580 | KEY=0 | LED=0 | TEMP_RAW=0x0C84 | TEMP=35.36 °C
[23:23:36] CNT=00067581 | KEY=0 | LED=1 | TEMP_RAW=0x0C84 | TEMP=34.94 °C
[23:23:36] CNT=00067582 | KEY=0 | LED=0 | TEMP_RAW=0x0CAA | TEMP=35.19 °C
[23:23:37] CNT=00067583 | KEY=0 | LED=1 | TEMP_RAW=0x0C7A | TEMP=34.82 °C
[23:23:37] CNT=00067584 | KEY=0 | LED=0 | TEMP_RAW=0x0C5A | TEMP=34.57 °C
[23:23:37] CNT=00067585 | KEY=0 | LED=1 | TEMP_RAW=0x0C8A | TEMP=34.94 °C
[23:23:37] CNT=00067586 | KEY=0 | LED=0 | TEMP_RAW=0x0C84 | TEMP=34.70 °C
[23:23:37] CNT=00067587 | KEY=0 | LED=1 | TEMP_RAW=0x0C32 | TEMP=34.33 °C
[23:23:37] CNT=00067588 | KEY=0 | LED=0 | TEMP_RAW=0x0C54 | TEMP=34.57 °C
[23:23:37] CNT=00067589 | KEY=0 | LED=1 | TEMP_RAW=0x0CEC | TEMP=35.68 °C
[23:23:37] CNT=00067590 | KEY=0 | LED=0 | TEMP_RAW=0x0C66 | TEMP=34.70 °C
[23:23:37] CNT=00067591 | KEY=0 | LED=1 | TEMP_RAW=0x0C7A | TEMP=34.82 °C
[23:23:38] CNT=00067592 | KEY=0 | LED=0 | TEMP_RAW=0x0C92 | TEMP=35.07 °C
[23:23:38] CNT=00067593 | KEY=0 | LED=1 | TEMP_RAW=0x0CC4 | TEMP=35.44 °C
[23:23:38] CNT=00067594 | KEY=0 | LED=0 | TEMP_RAW=0x0C86 | TEMP=34.94 °C
[23:23:38] CNT=00067595 | KEY=0 | LED=1 | TEMP_RAW=0x0C8E | TEMP=35.31 °C
[23:23:38] CNT=00067596 | KEY=0 | LED=0 | TEMP_RAW=0x0CAA | TEMP=35.19 °C
[23:23:38] CNT=00067597 | KEY=0 | LED=1 | TEMP_RAW=0x0C86 | TEMP=35.19 °C
[23:23:38] CNT=00067598 | KEY=0 | LED=0 | TEMP_RAW=0x0C8E | TEMP=35.19 °C
[23:23:38] CNT=00067599 | KEY=0 | LED=1 | TEMP_RAW=0x0C8E | TEMP=34.94 °C
  
```

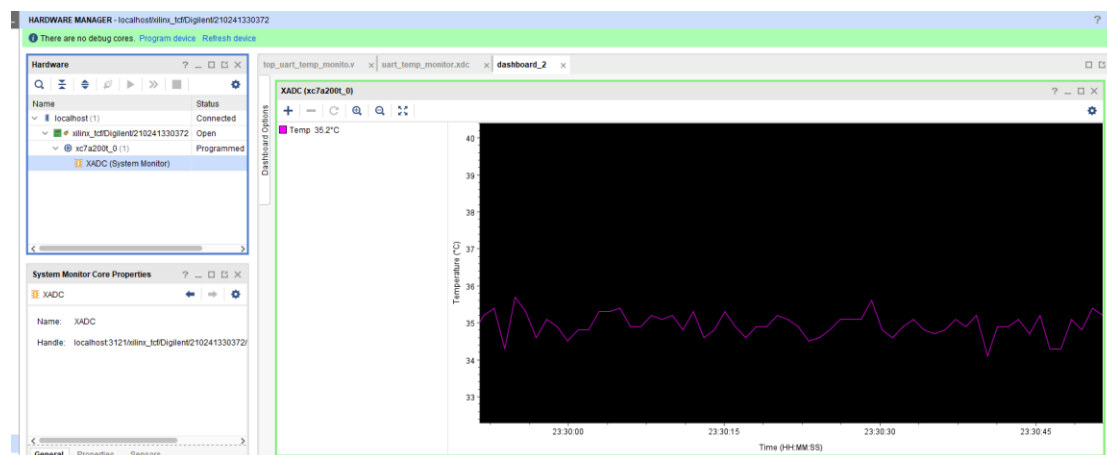
即時傳送 FPGA 內部溫度與板載狀態資料。按下按鍵，KEY 欄位會反映

輸入狀態；LED 欄位則隨心跳狀態週期性變化。

七、驗證與其他資料

(一)Vivado System monitor

Vivado 開啟 Hardware Manager 並連接開發板後，可使用 System Monitor 即時監控 FPGA 內部溫度，並與 Python 接收端換算結果進行比對。



(二)實驗系統規格

FPGA 晶片	AMD/Xilinx Artix-7 XC7A200T
系統時脈	50 MHz
感測模組	XADC Wizard IP
溫度來源	FPGA 內部 Temperature Channel
UART BAUD Rate	115200
UART 格式	8N1，無同位元
電腦端程式	Python + pyserial
輸出資料	CNT、KEY、LED、TEMP